

HOW MANAGING TECHNICAL DEBT CAN LEAD TO MORE SUSTAINABLE SOFTWARE

Lappeenranta–Lahti University of Technology LUT

Bachelor’s Programme in Software and Systems Engineering, Bachelor's thesis

2026

Bence Bánsághi

Examiner: Associate Professor Ari Happonen

ABSTRACT

Lappeenranta–Lahti University of Technology LUT

LUT School of Engineering Sciences

Software and Systems Engineering

Bence Bánsághi

How Managing Technical Debt Can Lead to More Sustainable Software

Bachelor's thesis

2026

36 pages and 3 tables

Examiner: Associate Professor Ari Happonen

Keywords: Technical Debt, Software Sustainability, Technical Debt Management, Green IT, Karlskrona Manifesto, Software Longevity

Digital architecture is a vital part of everyday life, making the long-term viability of software as crucial as physical infrastructure. This thesis investigates how the prioritization of technical debt serves as a critical part of achieving holistic software sustainability. While traditional research is divided between focusing on direct energy consumption with Green IT and focusing on development velocity and project budgets with Technical Debt Management, a significant gap exists in the literature connecting the two fields.

The research was conducted as a systematic literature review to identify, evaluate, and synthesize existing academic and industry sources, utilizing the five-dimensional sustainability model to analyse the impact of various debt types.

The results are presented through a Technical Debt-Sustainability Matrix, mapping 14 identified debt types against the five sustainability dimensions. The findings show that while technical and economic impacts are well-documented, environmental and human factors are often ignored in standard TDM frameworks. Specifically, infrastructure and configuration debt are linked to high environmental risks, while most debt types contribute to developer burnout and increased cognitive load, undermining individual sustainability. Furthermore, "Accessibility Debt" is identified as a social sustainability issue where some unmanaged choices can lead to the digital exclusion of marginalized users.

To address these gaps, the thesis proposes a Sustainability-Aware Prioritization Framework that redefines the interest of technical debt to include environmental waste, human fatigue, and the erosion of community trust. By integrating these multidimensional impacts into decision-making, organizations can transition from short-term feature delivery toward ensuring long-term viability and ethical responsibility of their software systems.

SYMBOLS AND ABBREVIATIONS

Abbreviations

AI	Artificial Intelligence
CI/CD	Continuous Integration and Continuous Delivery
CPU	Central Processing Unit
IEA	International Energy Agency
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
IT	Information Technology
LLM	Large Language Model
NFR	Non-Functional Requirement
OOD	Object-Oriented Design
QA	Quality Assurance
QA	Quality Assurance
ROI	Return-On-Investment
SE	Software Engineering
SS	Sustainable Software
SSDC	Software Sustainability Design Catalogue
TD	Technical Debt
TDM	Technical Debt Management

Table of contents

Abstract

Abbreviations

Declarations

1 Introduction 6
2 Literature review 8
2.1 The foundations of Technical Debt 8
2.1.1 The evolution of the metaphor 8
2.1.2 Types of Technical Debt other than Code Debt 9
2.1.3 Frameworks for Technical Debt Management and Prioritization 11
2.2 The Foundations of Software Sustainability 13
2.2.1 The Karlskrona Manifesto and the Five-Dimension Model 13
2.2.2 Analysis of the Five Dimensions of Sustainability 15
2.2.3 Practical Application of the Sustainability Model 16
2.2.4 Alternative “Lumper” and “Splitter” Models 17
3 Methodology 18
3.1 Research Approach 18
3.2 Literature Review Data Collection 18
3.3 Data Analysis and Framework Construction 19
3.4 Reliability and Limitations 19
4 Results and Discussion 20
4.1 The Technical Debt - Sustainability Matrix 20
4.1.1 The System Group 23
4.1.2 The Environmental Group 23
4.1.3 The Human Group 23
4.2 A Sustainability-Aware Prioritization Framework 24
5 Conclusions 26
5.1 Research Findings and Contributions 26
5.2 Limitations and Future Research 27

References

DECLARATIONS

Turnitin

The originality of this thesis has been reviewed with the Turnitin similarity checking service.

AI usage

The author of the thesis, Bence Bánsághi, used the following AI-tools during the preparation of the thesis:

1. Gemini Deep Research
    a. Purpose of use: Discovery and summarization of scientific papers.
    b. Explanation of the use of the tool: Used to conduct targeted searches of academic papers and use the results to help supplement the manually selected references and help support the existence of the thesis's independently identified primary research gap. The AI summary was only used to get a general idea of the papers' content and help determine relevant documents for manual review.
2. Gemini
    a. Purpose of use: Proofreading.
    b. Explanation of the use of the tool: Used iteratively to help confirm and correct sentence and paragraph logical flow, as well as get general thesis advice.
3. Grammarly plugin
    a. Purpose of use: Proofreading and improving writing.
    b. Explanation of the use of the tool: Used to check for grammatical errors, spelling, and punctuation.

Responsibility

The author, Bence Bánsághi, takes full responsibility for the content of this thesis and has reviewed and edited the content generated by the possible use of AI tools.

1 Introduction

Digital architecture is as much a vital part of, and a crucial aspect of, everyday life as physical infrastructure, and the long-term viability of software is also just as important as it is for its physical counterpart. Planning for the whole lifetime of the software has been an issue ever since its early days, with it coming up as an issue even in the first software engineering conference (Randell 1979). The term “sustainability” often carries with it an image of environmental conservation, sustainable technology use for environmental protection (Zhang & Li, 2022; Happonen & Ghoreishi, 2022), and tools for environmental protection (Huang et al., 2023; Eskelinen et al., 2017). It is, however, a much more encompassing concept, including things such as economic resilience, social well-being, and ease of long-term use. Fast development and demanding business conditions often force teams to take shortcuts, accumulating “technical debt” (Cunningham 1992), a metaphor for the implied cost of rework by choosing an easy solution now instead of using a better approach that would take longer initially. This thesis will demonstrate through a systematic literature review how the strategic prioritization of technical debt leads not only to financial or technical benefits but is also a critical building block of achieving holistic software sustainability. This topic is quite urgent right now due to the exponential growth of software’s energy consumption, especially with the rise of data warehouses and AI server farms (IEA 2025), and the high rates of developer burnout (Kaur et al. 2022).

The review of existing literature shows that there are mostly two directions of research in sustainable software engineering, one being Green IT and Sustainable Software Engineering (Becker et al., 2015). Strides have been made in areas such as optimizing data center energy efficiency (IEA, 2025), designing energy-aware algorithms (Kruglov & Succi, 2023), and measuring the power consumption of code (Făgărășan et al., 2023). These studies provide valuable tools to reduce the direct environmental footprint of software. The other main direction of research is in Technical Debt Management (Lenarduzzi et al., 2021; Graham et al., 2025), which mainly focuses on its detrimental effects on development velocity, code quality, and project budgets. There are various models for identifying and quantifying technical debt, focusing on its economic consequences and the immediate risk it presents to project timelines and maintainability.

However, a significant gap exists in the literature connecting Technical Debt Prioritization with holistic Sustainability Goals. The two research areas, Green IT and Technical Debt Management, are rarely connected. Research on technical debt prioritization, such as the work by Lenarduzzi et al. (2021), often focuses on the economic or technical project risks, while research on software sustainability, such as that by Kruglov & Succi (2023) and Fatima & Lago (2024), often focuses on high-level architectural design or direct energy use. The day-to-day management and strategic prioritization of existing technical debt as mechanisms for achieving broader sustainability goals beyond code quality or energy efficiency are not sufficiently examined in the literature.

This thesis aims to answer the following questions:

Q1: How does the existing literature define holistic software sustainability in relation to technical debt?

Q2: What frameworks can be used to prioritize technical debt based on its impact on sustainability factors?

Q3: How does the strategic prioritization of technical debt help achieve a more sustainable software product and development process?

The research scope is limited to a literature review of existing academic and industry sources, and does not involve data collection. Chapter 2 provides a review of the literature on technical debt and software sustainability. Chapter 3 shows the methodology for analyzing and connecting the two concepts. Chapter 4 discusses the findings and proposes a framework linking debt prioritization to sustainability. Chapter 5 provides conclusions, answers the research questions, and suggests directions for future research.

2 Literature review

This chapter is about conducting a literature review of the foundation of this thesis: Technical Debt (TD) and Software Sustainability (SS). It is made up of three sections. The first and second will review the foundations, origins, and existing metrics of measurements of TD and SS, respectively. The third section will synthesize the previous two chapters to identify the research gap.

2.1 The foundations of Technical Debt

The word “technical debt” as a metaphone at the original introduction by Cunningham (1992) was not meant to be pejorative. Instead, it was a communication tool used to explain a complex software quality trade-off to non-technical stakeholders who were more familiar with business concepts. It was used as an analogy to financial debt to describe the consequences of releasing less-than-perfect code to meet a short-term goal. He argued that it was acceptable to “borrow” time by taking some shortcuts, but this led to the buildup of a “loan” that would have to be “repaid” later through refactoring. The “interest” on this debt would be the “extra development effort required in the future”, which, if left to accumulate, would cause serious problems and may reach a point where an “entire engineering organization can be brought to a stand-still”. A fitting analogy would be road maintenance, where skipping maintenance saves some money in the short term, but as time goes on, more and more is needed as the road worsens. If the problem is left to fester and not taken care of for a long time, it can break down to the point of unusability and lead to deadly accidents and huge ramifications.

2.1.1 The evolution of the metaphor

Fairbanks (2020) terms “ur-technical” debt, which refers to the original concept as proposed by Cunningham, as an inevitable and sometimes prudent consequence of iterative development where the team’s understanding of the problem necessarily evolves. This is a sharp contrast to the modern usage of TD, which is often used for broader and less specific

concepts, such as “hacky code”, “code written by novices”, or systems that had devolved into a “big ball of mud”.

To clear up the ambiguity, Fowler (2009) proposed the “Technical Debt Quadrant”, a TD categorization framework that uses two dimensions: the intent (Deliberate vs Inadvertent) and the context (Prudent vs Reckless). This distinguishes debt into four sections:

Prudent and Deliberate: A conscious and strategic decision. This aligns with Cunningham’s original concept of “We need to go live now and will fix it later”.

Prudent and Inadvertent: The “ur-technical-debt” described by Fairbanks, where a team realises “Now we know how we should have built it”.

Reckless and Deliberate: A team ignores the best practices under pressure, with no plan of remediation.

Reckless and Inadvertent: A lack of knowledge or awareness of best practices.

This thesis adopts the stance that while all quadrants represent TD, the management and prioritization frameworks are most relevant in the Prudent and Deliberate quadrant, as that is the section where strategic trade-offs are deliberately made, and where their effects are likely to be taken into account before committing to it.

2.1.2 Types of Technical Debt other than Code Debt

To manage TD effectively, one must look beyond simple poor code quality. While “Code TD” is the most frequently used, tooled, and encountered problem, it is only one component of a systematic problem. For instance, decisions made at the architectural level can negatively impact long-term system health and evolvability, while suboptimal hardware or deployment scripts can introduce severe infrastructure and configuration debt.

Table 1 shows the expansive typology of debt, identified from the literature of Alves et al. (2014), Rios et al. (2018), Verdecchia (2021), and Caglayan & Özcan-Top (2024). This classification demonstrates that throughout the software lifecycle, TD is present at every step.

Table 1: Types of Technical Debt

Debt Type	Definition	Example
Architecture debt	Decisions at the architectural level that negatively impact long-term system health, maintainability, and evolvability.	Using monolithic architecture for a system that requires high modularity and independent scaling.
Build Debt	Build-related issues that make the build process harder and more time/processing consuming.	The build process needs to run ill-defined dependencies, making the process slow.
Code Debt	Problems in the source code that affect maintainability and legibility.	Bad coding practices, such as copy-pasting code instead of creating reusable functions.
Configuration Debt	Debt in the configuration files and deployment scripts.	Hard-coding credentials in multiple config files instead of using a central service.
Defect Debt	A known and low-priority defect left in the backlog to be fixed later, commonly referred to as ”bugs”.	Any issue that comes up under expected usage, stemming from bad code.
Design Debt	Faster short-term design choices that violate OOD and make future work cost more to complete.	Creating a class that handles too many functions, making it difficult to modify or test.
Documentation Debt	Missing, incomplete, or outdated documentation makes onboarding, understanding, or maintenance harder.	Modifying an API’s parameters without updating the public documentation, confusing users.
Infrastructure Debt	Outdated or suboptimal hardware, operating systems, or CI/CD pipelines that create scalability and reliability issues.	Relying on manual deployment processes or using an older version of software for hardware compatibility.
Management Debt	Suboptimal decisions related to project management, often caused by inappropriate planning and ineffective project management.	Prioritizing new features over refactoring, allowing the debt to accumulate to unmanageable levels.
People Debt	Deficiencies in team knowledge or skills, often stemming from a lack of qualified professionals.	A team of junior developers, or a ”10x developer” (Flournoy et al., 2025), who alone understands critical subsystems.
Requirements Debt	Suboptimal decisions made during the gathering, formalization, or implementation of requirements.	Ending up with ambiguous specifications that leave out key metrics, forcing the developers to guess.
Service Debt	Compromises made during the design, implementation, or integration of microservices and services.	Too broad or over-segmented services, leading to a central bottleneck, or creating high-latency dependencies.
Test Automation Debt	Debt from a poorly designed, brittle, or slow automated test suite, which may become a bottleneck.	A test suite where some tests fail randomly or take hours to run, leading to developers not trusting it or skipping it altogether.
Test Debt	The consequence of postponed or inadequate testing that results in gaps in test coverage.	Only testing for probable usage, leaving edge cases and failure modes untested.

While a single, universally standardized ontology is still undergoing academic refinement, a general consensus has emerged from mapping studies, with the most widely accepted consolidation study done by Li et al. (2015), who identified ten core types of debt, with Code, Design, Architecture, Test, and Documentation being the most prominent ground

values. Table 1 contains some amount of overlap between the categories and is neither concise nor exhaustive, but it tries to expand the types of debt that are emerging to reflect the newer studies.

A significant finding in literature examining the real-life implementation of academic knowledge is an “academic-practitioner typology gap”. While studies identify this broad array of debt types, industry tools and practices “focus on code and architectural debt, whereas documentation and process debt remain largely unaddressed” (Terrón-Macias et al., 2025). This lack of proper care can be said for most of the rest of the categories described above, which seems to suggest that organizations are likely to manage only the most visible and easily measured forms of debt, while incurring significant, unmonitored liabilities in other, equally important areas.

2.1.3 Frameworks for Technical Debt Management and Prioritization

There exists a wide range of coverage on Technical Debt Management (TDM), ranging from identification and measurement to repayment. The most critical and least mature of these activities is debt prioritization, which is concerned with the repayment and deferral of debt. Lenarduzzi et al. (2021) provide a comprehensive overview of the field, and their findings are the following: prioritization research is considered “preliminary”, there is “no consensus on what the important factors are and how to measure them”, and that “there is a lack of empirical and validated set of tools”.

Given that even medium-sized codebases can contain “thousands of technical debt items”, it is “infeasible to pay off all items” (Pina, 2023). To navigate the complexity and the sheer amount of debt, a study by Ribeiro et al. (2016) identified 14 decision-making criteria, with the two most common being “debt impact on the project” and “cost-benefit”. While there is no single unified model, there are three main prioritization model groups: economic models, project-centric risk models, and business-driven value models.

Economic (Return-On-Investment (ROI)) based models are based on cost-benefit analysis. For it to be effective, the metaphor must be translated into actual, quantifiable terms. Nugroho et al. (2011) provide the foundational model for this translation and define the two key components: Principal, defined as “the cost of repairing quality issues [...] to achieve an

ideal quality level", meaning the one-time investment required to pay back the debt, and Interest, defined as "the extra cost spent on maintenance due to technical quality issues", which is the recurring additional effort spent because of the existence of the debt. This model achieves prioritization through a clear ROI calculation. A manager can "invest" the Principal to eliminate the future Interest payments, so debt items with the highest interest-to-principal ratio are prioritized.

Project-centric (risk-based) models prioritize debt based on its concrete and measurable impact or risk on the project's health, its developers, and its quality, using categorizations such as "critical, major, normal, and minor" (Akbarinasaji et al., 2017). The impact can be quantified in relation to maintainability, as debt degrades "design-time qualities" (Avgeriou et al., 2023). For example, Zazworka et al. (2011) found that "god classes" are "changed more often; and … contain more defects" than others, suggesting a strategy of prioritizing debt in components that are frequently changed. Another way of quantifying is with the impact on development velocity. This is one of the most visible symptoms of TD, because "implementing workarounds for issues in the code can take significant time" (Tom et al., 2013).

Economic and Project-centric models are not only not contradictory, but they are two sides of the same coin. The economic interest is not an abstract figure, but a monetary manifestation of wasted developer time that could have been better spent on productive tasks. Similarly, the interest payment can be equated to the time spent on hard-to-navigate code segments, fixing maintainability issues, or waiting on long testing or building processes.

A critical gap in the literature is that both the economic and project-risk models often fail to consider high-level business strategy, which has led to the development of business-driven frameworks, of which the most prominent is the "Tracy" framework (Almeida et al. 2019). Its main innovation is a mechanism for linking low-level technical artifacts to high-level business value, which creates a clear map of dependencies; Technical Debt Items, which affect Configuration Items (such as modules or classes), which support IT assets (such as the software product), which enable Business Processes (such as "inventory management") (Almeida et al., 2021). With this map, prioritization is not based on the technical severity or ROI of the debt item, but on the business value of the "Business Process" it ultimately

affects. This value maximization approach moves the objective from finding the most costly debt item to finding the debt item that is blocking the most valuable business processes.

While these frameworks represent a clear advancement from simple technical prioritization to sophisticated, business-aligned strategies, they all have a limitation; they are all focused on economic, project, or business values, and do not take sustainability values into account. The interest is calculated only in developer-hours or project delays, but not in the environmental cost of the energy-inefficient code, architecture, infrastructure, build process, or other TD item. The risk is defined as a project failure or security breach, but not as the long-term social cost of developer burnout or the decay of the system's long-term viability. This narrow view of TDM is the primary research gap this thesis addresses. The next section will, therefore, review software sustainability to review these missing dimensions.

2.2 The Foundations of Software Sustainability

To bridge the research gap identified in the previous section, it is necessary to define "software sustainability" and explore its origins. While "the concepts of sustainable and green are often used interchangeably, … sustainability includes green and it goes beyond green" (Oyedeji, 2018), and this thesis uses the concepts in this way. Software sustainability thus evolved from "Green IT", or "Green Software", which focused exclusively on the environmental dimension. It was concerned with measurable, direct impacts, such as the energy consumption of data centres and of code, carbon footprints, and waste and resource management. This perspective, while valuable and indispensable, was incomplete and appears to be a symptom of quantification bias, where only the easily quantifiable environmental factors were taken into account. On the other hand, complex human factors such as developer well-being, which are hard to measure, were ignored by the initial wave of research.

2.2.1 The Karlskrona Manifesto and the Five-Dimension Model

The shift from the initial green-only definition of Sustainable Software to the more inclusive one was formalized by academics who researched and developed socio-technical analysis methods. The most influential work to do so was the Karlskrona Manifesto for Sustainability

Design (Becker et al., 2015), which was designed to be “a common ground and a point of reference” to combat the “misperceptions” and “lack of coherent understanding” of sustainability and its relations to software. It argued that sustainability is not just one more non-functional requirement (NFR), but a fundamentally different way of viewing the field’s value system. The choice of the Manifesto genre was deliberately made to try to invoke feelings similar to those of the Agile Manifesto (Beck et al., 2001) and to place itself on the same level of value statement it brought to SE. The intent of reorientation away from “comprehensive documentation” and towards “individuals and interactions” in the Agile Manifesto can be equated with the intent to reframe sustainability from merely another NFR to a “first-class concern” that must be present in every activity, in the Karlskrona Manifesto.

For the purposes of this thesis, the Manifesto’s most crucial part is the adoption and popularization of a five-dimensional model for sustainability, which can be traced back to Penzenstadler & Femmer’s (2013) work, which provides the definitions for a truly holistic analysis. The five dimensions are the following: Individual, Social, Economic, Environmental, and Technical. Their details can be found in Table 2.

Table 2: The Foundational Five-Dimension Model for Sustainability and its Software Engineering Interpretations (Penzenstadler & Femmer, 2013 / Becker et al., 2015)

Dimension	Core Definition	Key SE Question	Key Concepts and Metrics
Individual	”Maintenance of the private good of individual human capital (health, education, skills, knowledge, leadership, access to services).”	”How can software be created and maintained...that enables developers to be satisfied with their job over a long period of time?”	Developer well-being, knowledge management, team health, ”Sustainable Personality”
Social	”Maintaining social capital and preserving the societal communities in their solidarity. Social capital...create[s] the basic framework for society.”	”Which effects do software systems have on the society (e.g., communication, interaction, government, etc.)?”	Equity, inclusion, accessibility, societal trust, digital divide, value-based requirements.
Economic	”Maintaining assets. Assets comprise not only capital but also added value...protect[ing] stakeholders' long-term investments...from economic risks.”	”How can software systems be created so that the stakeholders' long-term investments are as safe as possible from economic risks?”	Total Cost of Ownership, Net Present Value, Value-Based SE, long-term viability.
Environmental	”Improve human welfare by protecting natural resources (water, land, air, minerals, and ecosystem services)...ensure...sink capacities...are not exceeded.”	“How does software affect the environment during, inter alia, development and maintenance?”	Energy consumption, carbon footprint, e-waste, 1st/2nd/3rd-order effects.
Technical	”Long-time usage of systems and their adequate evolution with changing surrounding conditions and respective requirements.”	”How can software be created so that it can easily adapt to future change?”	Maintainability, evolvability, adaptability, longevity, TDM.

2.2.2 Analysis of the Five Dimensions of Sustainability

For ease of understanding and explanation, this thesis will group the five dimensions of sustainability into three separate thematic groups: the Environmental Dimension as a standalone, the Human group, comprised of the Individual and Social Dimensions, and the System group, comprised of the Economic and Technical Dimensions. The groupings are introduced to achieve two goals: Human is meant to analyze the human-centric factors, and to show how the original "three pillar" model's non-separation of them was lacking. The System grouping is made to help demonstrate the central argument that Technical Debt Management is focused almost entirely on technical and economic factors.

The Environmental Dimension remains as a standalone category within this analysis, falling directly under the "Greet IT" umbrella, and it can broadly be described as "protecting natural resources" (Penzenstadler & Femmer, 2013). It provides the academic background for environmental metrics like Energy Debt and Ecological Debt, which are "quantified but not prioritized". The Environmental Dimension is directly impacted by systematic TD, such as energy-inefficient Architecture Debt, resource-heavy Infrastructure Debt, or computationally expensive Build Debt.

While the Individual and Social dimensions are distinct within the Five-Dimension model, for the purposes of this thesis, they can be grouped together as the Human grouping. The separation between them is, however, still important to have, as not only do these two dimensions not necessarily imply one another, but they can be independently fulfilled or contradict each other. For example, a non-profit project may have high Social sustainability by promoting social equity, but the developers can be underpaid and overworked, which would imply a low Individual sustainability score. And so, Individual Sustainability is inward-looking, and is the basis for developer well-being and knowledge sustainability. A high Individual Sustainability score can be derived from low People and Documentation Debt. On the other hand, Social Sustainability focuses on social capital and is concerned with equity, inclusion, and accessibility for groups of people and society at large. This separation is key to a truly inclusive analysis.

The Technical and Economic Dimensions can be looked at together to make up the System grouping, as their main concerns are with the long-term viability of the software system itself, and they link directly back to TDM. Maintainability, evolvability, and the direct

management of TD are linked with Technical Sustainability, as its definition is about the long-term usage and evolution of systems. Economic Sustainability's main goals are to maintain assets, grow value, and ensure the software is not under duress from economic risks using ROI and Cost-Benefit Analysis models.

These groups are not isolated; the Human and Economic groups share a critical link together, as a failure to fulfil Technical Sustainability leads to a hard-to-maintain system, which in turn causes failures in Economic Sustainability, as more resources are spent on maintaining it than developing new features. This causes problems in Individual Sustainability, as the team is more and more focused on irritating bug fixes, leading to burnout.

2.2.3 Practical Application of the Sustainability Model

The Karlskrona Manifesto provided the high-level principles, but no practical application. There has, however, been quite substantial subsequent research as to how it can be applied to actual engineering practice. The most prominent research is the Software Sustainability Design Catalogue (SSDC) (Oyedeji et al., 2018). The SSDC's stated purpose is to "bridge the current gaps between the principles and the indicators of sustainability" by providing "concrete guidelines". It achieves this by mapping the Manifesto's principles onto 8 phases of the Software Development Life-Cycle. For each phase, it proposes specific Sustainability Goals and measurable Indicators, for example, "design […] [to] educate users about sustainability" in the Implementation phase.

This progression from principles to practice has continued to mature from SSDC to more specialized and formalized tools. Moreira et al. (2023) developed a "social and technical sustainability requirements catalogue", which specializes in two dimensions (Social and Technical), and argue that using the catalogue leads to a reusable context window for the developer that results in the overall identification and development of a more sustainable system. Restrepo et al. (2024) developed "SinSO: An ontology of sustainability in software", which, as an ontology, strives to represent formal knowledge. It aims to "contribute to reducing ambiguity and enhancing comprehension" in the field of SS, further developing the relationship between the five dimensions.

2.2.4 Alternative “Lumper” and “Splitter” Models

While the five-dimension model is the dominant one, it is not unchallenged. There is a divide between academics who wish for higher practicality and its consequence of fewer dimensions (“Lumpers”), and those who wish for theoretical completeness and argue that the 5 dimensions are insufficient (“Splitters”).

An example “Lumper” model is one proposed by Condori-Fernandez et al. (2020) that condenses, or more appropriately, subsumes the Individual dimension into Social. The main advantage of this is its advanced practicality, as it manages to directly map its dimensions onto existing industry standards, like the ISO/IEC 25010 software quality attributes. This simplification, however, loses the “Individual vs. Social” analysis.

The ShapeRE framework proposed by Pham et al. (2020) approaches the dimensional question from the exact opposite side, splitting it into 9 dimensions by adding Purpose, Design-Aesthetic, Integrative, and Legal dimensions on top of the original 5, taken from the field of building architecture. While the authors acknowledge that “all dimensions may not be relevant for all requirements of a system and may vary”, the argument is that “it is important to consider the complete set of dimensions […] to increase the completeness at the cost of a complex system view”.

The ongoing debate reinforces the centrality of the 5-dimension model, as most studies take it as the foundation and expand or simplify from it. For the purposes of this thesis, the expanded or simplified models are not considered by the author, as the base model is felt to provide the ideal balance and established presence in the field.

3 Methodology

This chapter outlines the research methodology used to answer the research questions defined in Chapter 1. As the aim of this thesis is to bridge the gap between the fields of Technical Debt Management and Software Sustainability, which have been traditionally studied separately, a qualitative research based on a systematic literature review was chosen.

3.1 Research Approach

The primary method used in this study is a systematic literature review. This approach allows for identifying, evaluating, and synthesizing existing research to create a new conceptual framework. It follows a defined protocol to reduce bias and ensure that the findings are reproducible and comprehensive. The goal of this methodology is not only to summarize existing papers, but to create a framework that combines the two fields of research, specifically, to map the known types of technical debt to the five dimensions of sustainability.

3.2 Literature Review Data Collection

The data collection was conducted in separate phases. Firstly, two main databases, Google Scholar and LUT Primo, were considered, and broad searches were conducted to identify relevant papers to begin the research. The search strings were built from combining the various fields' key relevancies, such as technical debt, design debt, code debt, debt prioritization, debt management, sustainability, green software, social debt, energy efficiency, and maintainability. Relevant papers were selected for closer review, and a baseline understanding of the current state of the fields was established.

Papers were included based on their relevance to the topics, especially if they covered both main fields, and they were prioritized based on the number of citations and publication date. Only peer-reviewed journal articles, conference proceedings, and reputable industry papers written in English were taken into account. Preference was given to newer papers to ensure relevance, except for foundation papers on the topics (such as Cunningham, 1992).

3.3 Data Analysis and Framework Construction

The analysis of the collected literature was performed using a qualitative thematic analysis approach. The method was chosen for the identification of recurring patterns in the way different types of debt impact various parts of software development. First, the literature on technical debt was analysed to categorize debt into distinct categories, as seen in Table 1. The same process was also applied to Software Sustainability to define the five core dimensions, presented in Table 2. Then, Technical Debt types were evaluated against the Sustainability Dimensions, and finally, the findings were synthesized into a conceptual framework

3.4 Reliability and Limitations

The reliability of this research is based on the use of established academic databases and peer-reviewed sources; however, being a literature-based study, this thesis has certain limitations. It relies on secondary data rather than empirical testing, and the conceptual framework is theoretical and has not been validated through an industrial case study or experiment. Secondly, while the search was systematic, there is a potential for selection bias, as the intersection of the two fields is a niche field, and relevant papers using non-established terminology might have been missed. Finally, the rapid evolution of AI and software infrastructure means that older sources regarding energy consumption may be outdated and less-applicable to modern contexts.

4 Results and Discussion

This chapter presents the findings of the literature review and answers the research questions regarding the connection between TD and SS. The core result is a matrix that visualizes how specific identified types of debt negatively impact the five dimensions of sustainability. Following the matrix, an analysis is presented grouped by the “Environmental”, “Human”, and “System” categories established in Chapter 2.

4.1 The Technical Debt - Sustainability Matrix

To answer Research Question 2 ("What frameworks can be used to prioritize technical debt based on its impact on sustainability factors?"), a cross-impact matrix was constructed. This matrix (Table 3) maps the fourteen types of Technical Debt identified in the literature to the five dimensions of sustainability. Many links to debt types in the referenced papers were deduced, as they were not directly stated. Their impact, based on the author’s opinion, is showcased in the following way: Critical, High, Medium, and Low/No evidence found in literature.

Table 3: Technical Debt - Sustainability Matrix

Debt Type	Technical	Economic	Environmental	Social	Individual
Architecture	Architectural erosion undermines evolvability and long-term viability (Besker et al., 2018)	Needs a massive budget for maintenance, up to 40% of the total budget (Trova, 2024)	Monolithic rigidity and poorly distributed architecture lead to higher network energy consumption (Zhao et al., 2025)	No direct link identified	Architectural workarounds significantly increase cognitive load and developer frustration over time (Besker et al., 2020)
Build	Slows feedback loops and causes fragile dependencies (Ghammam et al., 2025)	Increased computation costs and wasted CI/build/test resources (Morgenthaler et al., 2012)	Wasted energy on redundant and failed CI/CD and compilation (Saavedra et al., 2025)	No direct link identified	Lower engineering productivity due to slower builds and brittle targets (Morgenthaler et al., 2012)
Code	Increases entropy and reduces	Increases the cost of change, reduces ROI,	Inefficient algorithms increase	No direct link identified	Developer demoralization and cognitive

	maintainability (Conejero et al., 2018)	and slows time-to-market (Conejero et al., 2018)	CPU/energy usage (Marantos et al., 2020)		fatigue (Besker et al., 2020)
Configuration	Deployment reliability and evolvability is diminished (Rahman et al., 2019)	Causes outages and operational failures (Xu and Zhou, 2015)	Inefficient autoscaling causes high energy usage (Alharthi et al., 2024)	No direct link identified	High cognitive load switching between environment disparities (Miller, 2025)
Defect	”Broken window” effect (Levén et al., 2024) lowers quality; defects lead to instability and cascading failures	Compounding interest makes fixing defects exponentially more costly over time (Dawson et al., 2010)	No direct link identified	Leaving known defects unresolved can undermine user trust over time (Baughan et al., 2023)	No direct link identified
Design	Harms maintainability through increased coupling and reduced modularity (Lenarduzzi et al., 2021)	Increases long-term cost by raising maintenance effort and reducing productivity (Guo et al., 2014)	Design choices affect energy consumption via changes in runtime behavior (Noureddine & Le Goaer, 2025)	No direct link identified	Increased mental load by making changes non-local and reasoning harder (Olsson et al., 2021)
Documentation	Critical pieces of missing documentation make systems harder to maintain and evolve safely (Garousi et al., 2013)	Increases maintenance costs due to more time and effort required for fixing issues (Garousi et al., 2013)	No direct link identified	Customer-facing debt can reduce user or community trust if they are misled or if critical information is not present (Silva et al., 2023)	Increases mental load by forcing developers to get important pieces of information either from code or peers (Garousi et al., 2013)
Infrastructure	Increases maintenance efforts and slows software evolution (Molnar & Motogna, 2020)	Raises maintenance and operation costs (Molnar & Motogna, 2020)	Inefficient and outdated infrastructure increases energy consumption (Ahmadisakha & Andrikopoulos , 2024)	Outages and degraded service availability lower user trust (Hou & Jansen, 2022)	Increases developer stress through outdated CI/CD, environments, and operational interruptions (Besker et al., 2020)
Management	Planning and management issues are strongly related to the causes of technical debt (Berenguer et al., 2023)	Systematic deferral of maintenance creates compounding ”interest” (Guo et al., 2014)	Can indirectly lead to higher emissions by choosing not to prioritize energy-efficient refactoring work	Bad management can lead to a bad reputation for the software producer (Hou & Jansen, 2022)	Time pressure and the need to rely on shortcuts contribute to developer stress and bad morale (Besker et al., 2020)

People	Increases the chances of bad design and implementation decisions (Heldad et al., 2024)	A shortage of skilled employees can lead to low productivity and missed revenue goals (Ashare, 2024)	No direct link identified	No direct link identified	Causes significant problems with collaboration, communication, and coordination (Dayan-Akman et al., 2025)
Requirements	Requirements debt in relation to the five dimensions can mostly be generalized impact-wise, as any of them not being taken into account or deprioritized leads to less focus on it during the whole lifecycle of the software, leading to less sustainable practices altogether				
Service	Undermines evolvability by increasing coupling and reducing the benefits microservices are meant to provide (Borowa et al., 2025)	Increases costs by raising operational overhead and integration complexity (Borowa et al., 2025)	No direct link identified	Reliability issues can arise from service debt, leading to community trust erosion (Chhonker & De Lemos, 2019)	Granularity and integration issues increase developer cognitive load by complicating debugging and coordination (Borowa et al., 2025)
Test Automation	Unreliable automated tests undermine QA, allowing more bugs to slip into production, and can lead to lower code confidence (Gruber & Fraser, 2022)	A need for frequent rewrites and manual intervention leads to unnecessarily spent developer hours (Gruber & Fraser, 2022)	No direct link identified	No direct link identified	Randomly failing automated tests or breakage due to bad design causes developer experience to be less pleasant and more stressful (Gruber & Fraser, 2022)
Test	Lack of testing reduces fault detection, increasing defect density and lowering system maintainability (Boehm & Basili, 2005)	Postponing testing significantly increases defect-fixing costs later in the lifecycle (Boehm & Basili, 2005)	No direct link identified	No direct link identified	Insufficient testing leads to reactive debugging and increased stress (Andersson, 2022)

We can see that the impact of the varying types of TD is not evenly distributed across the five dimensions or the different debt types. The following sections analyse the impacts based on the thematic groupings established in Chapter 2.

4.1.1 The System Group

The matrix reveals that the System group is the most universally documented, and almost all identified debt types in this group are marked as ‘high’ or ‘critical’. For instance, Architectural, Design, Management, and Service debts are shown to have a ‘critical’ impact on technical maintainability and evolvability. Economically, these debts manifest as massive maintenance budgets (consuming up to 40% of the total budget in the case of architectural debt) and compounded interest that makes deferred maintenance exponentially more costly over time. There are no gaps where a direct link could not be identified or where the impact is low. This is, of course, due to the fact that technical debt management was established first and foremost to help with project-centric risk and ROI, however the matrix demonstrates that while these impacts are really important, treating them in isolation ignores the other three dimensions, undermining parts of software longevity.

4.1.2 The Environmental Group

Compared to the System group, the environmental aspect has a lower but highly specific concentration of impact. The literature links environmental degradation most strongly to systematic inefficiencies. For example, Configuration and Infrastructure debts have a high environmental risk, as inefficient autoscaling and outdated hardware increase network energy consumption and emissions. Architecture and Code debts also impact this dimension through rigidity and expensive algorithms. The human-centric and process-oriented debts show no direct links to direct environmental impact in the current literature. This is due to environmental impacts being heavily researched, where they can be measured in CPU cycles or kilowatt-hours, but indirect environmental costs, such as those of poor team practices, remain unexplored.

4.1.3 The Human Group

The matrix highlights a huge contract within the human grouping. The Individual dimension is heavily impacted across the board, but the Social dimension shows the most significant gap in current research. Individual sustainability, primarily focused on developer well-being,

cognitive load, and morale, is consistently compromised by TD. People debt is marked as critical, causing significant collaboration and coordination breakdowns, and most other types, particularly Code, Architecture, and Documentation debt, cause cognitive fatigue, frustration, and mental load. On the other hand, the Social dimension has the highest number of "No direct link identified" entries; however, a targeted review of specialized research reveals that this is more of an academic blind spot than a real-world one. Historically, "social debt" in software engineering has been inwardly focused on the development team, examining things such as "community smells" and internal communication friction rather than external societal harm (Cabllero-Espinosa et al., 2022).

Emerging research on social sustainability debt, however, explicitly includes inclusive design and equity, when accessibility is treated as a late-stage afterthought rather than an immutable architectural constraint. This debt type, known as "Accessibility Debt" (Suvvari, 2023), behaves exactly like compounding financial debt, making post-launch costs exponentially more expensive to pay back than considering it from the beginning. There is also a direct link between unmanaged architectural debt in critical public and educational infrastructure and the worsening of the divide. Outdated systems inherently discriminate against vulnerable populations, such as older adults attempting to access government services or students relying on low-bandwidth connections, essentially locking them out of digital ecosystems (Sahni et al., 2025). Current TDM frameworks fail to recognize that prioritizing rapid agile velocity over foundational code accessibility directly shifts these problems onto marginalized end-users.

4.2 A Sustainability-Aware Prioritization Framework

To answer Research Question 3, "How does the strategic prioritization of technical debt help achieve a more sustainable software product and development process?", it is necessary to move beyond the traditional TDM frameworks. The current models prioritize debt based primarily on ROI and immediate system risk. However, as demonstrated by the Sustainability Matrix, this leaves critical sustainability dimensions unmanaged. To achieve true holistic sustainability, a Sustainability-Aware Prioritization Framework must be adopted. This framework proposes expanding the traditional evaluation process through the following mechanisms:

- Mapping the Impact across all Dimensions: Before prioritizing a debt item, teams must assess its impact across all dimensions, making sure none of them remain unaccounted for. For example, while a piece of infrastructure debt might have a ‘medium’ economic impact, its reliance on outdated and energy-inefficient hardware elevates its environmental impact to ‘high’.

- Redefining “Interest” in ROI calculations: Nugroho et al.’s (2011) model defines “Interest” as the extra cost spent on maintenance. A sustainability-aware framework expands this definition to account for energy waste (environmental), cognitive fatigue and developer burnout (individual), and the erosion of community trust and accessibility (social). Debt items that compound interest across multiple dimensions must be prioritized higher.

- Elevating Human and Environmental urgency: Traditional business-driven frameworks prioritize resolving debt that blocks the most valuable business processes, which often fail to consider the long-term human or environmental cost of the debt being deferred. However, the matrix reveals that debt, such as people debt and defect debt, often cause severe coordination breakdowns or undermine user trust. By prioritizing documentation or code debt, developer burnout can be reduced, thereby securing the individual sustainability necessary to maintain the system long-term.

Ultimately, integrating these mechanisms aims to shift the organizational mindset from short-term feature delivery to ensuring long-term viability of the software system itself from all viewpoints of sustainability. By adopting this multidimensional approach, management of TD prevents cascading failures, and the repayment strategy creates an environment that balances economic resilience, social well-being, environmental protections, and ease of long-term use.

5 Conclusions

This thesis looked at the intersection of Technical Debt Management (TDM) and Software Sustainability (SS) to determine how debt prioritization contributes to long-term software viability. The three research questions have been successfully addressed by synthesizing existing literature into a sustainability-aware prioritization framework.

5.1 Research Findings and Contributions

The study identified that while traditional TDM usually focuses on the Technical and Economic dimensions, it only looks at the developer side of the Individual dimension, and largely neglects the Environmental and Social dimensions. The core contribution of this work is the Technical Debt-Sustainability Matrix, which maps 14 types of debt to the five dimensions of sustainability. This matrix demonstrates that environmental sustainability is critically impacted by Infrastructure and Configuration debt through energy-inefficient resource allocation, individual sustainability is degraded by nearly all forms of debt, leading to developer burnout and cognitive fatigue, and social sustainability, which directly contributes to the digital divide, largely remains an academic blind spot in the context of technical debt management. These findings coincide with the Karlskrona Manifesto, which states that sustainability must be a first-class concern in software engineering, rather than a secondary non-functional requirement. The proposed framework offers a practical evolution of the ROI model by redefining interest to include environmental waste and social exclusion. By adopting this framework, an organisational shift towards holistic software longevity from short-term delivery can be achieved. This is particularly urgent with the rising energy demands of AI and data centres.

On a broader scale, this research highlights the need for a shift in how the software industry views its products and its people. Historically, software engineering often operated on a short-term borrowing mentality, where the payment was assumed to be strictly financial or technical. However, integrating technical debt with the five-dimensional sustainability model reveals that the true cost of unmanaged debt is far more systemic. Prioritizing short-term feature delivery over holistic sustainability not only worsens code but also actively

harms developer well-being, contributes to environmental degradation, and risks digitally excluding marginalized users. True software sustainability requires recognising that code is not written in a vacuum, and consideration for the other dimensions of sustainability aside from technical and economic is essential.

5.2 Limitations and Future Research

The primary limitation of this thesis is its systematic literature research nature, as it lacks empirical validation through industrial case studies. Furthermore, the rapid evolution of AI infrastructure may render the cited energy consumption data outdated quickly. To build upon the theoretical foundation established in this thesis, future research should focus on transitioning from academic classification to practical industrial applications. The proposed Sustainability-Aware Prioritization Framework requires empirical validation, and a highly valuable follow-up study would involve research with an active software development environment, where the framework would be integrated into a team's Agile backlog processes. Observing how developers and product owners negotiate trade-offs between explicitly quantified environmental and human factors would provide crucial insight into the framework's real-world viability and points of improvement.

Secondly, while technical and economic debts are easily caught by automated tools like SonarQube, social sustainability remains unquantifiable in current pipelines. Future work should focus on developing measurable indicators for Accessibility Debt that would allow teams to track social exclusion risks to the same extent as they do code smells, thereby preventing accessibility from being treated like an afterthought. Finally, the exponential growth of LLMs and AI remains a not-yet fully explored area of TDM. As infrastructure becomes more and more computationally expensive, future research must investigate AI-specific technical debt, which includes studying how relying on black-box models impacts system evolvability, and the immense energy demands of training and querying these models. Managing the debt of AI systems will likely be a major challenge in software sustainability in the upcoming years.

References

Akbarinasaji, S., Bener, A., and Neal, A. (2017). A Heuristic for Estimating the Impact of Lingering Defects: Can Debt Analogy Be Used as a Metric? doi:https://doi.org/10.1109/wetsom.2017.7.

Alharthi, S., Alshamsi, A., Alseiari, A. and Alwarafy, A. (2024). Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions. Sensors, [online] 24(17), p.5551. doi:https://doi.org/10.3390/s24175551.

Almeida, R.R., do Nascimento Ribeiro, R., Treude, C. and Kulesza, U. (2021). Business-Driven Technical Debt Prioritization: An Industrial Case Study. [online] IEEE Xplore. doi:https://doi.org/10.1109/TechDebt52882.2021.00017.

Almeida, R.R., Treude, C. and Kulesza, U. (2019). Tracy: A Business-Driven Technical Debt Prioritization Framework. arXiv (Cornell University), 1904, pp.181–185. doi:https://doi.org/10.1109/icsme.2019.00028.

Alves, N.S.R., Ribeiro, L.F., Caires, V., Mendes, T.S. and Spinola, R.O. (2014). Towards an Ontology of Terms on Technical Debt. 2014 Sixth International Workshop on Managing Technical Debt. doi:https://doi.org/10.1109/mtd.2014.9.

Andersson, S. (2022). The impact of technical debt on stress A case study of a large Swedish company. [online] Available at: https://www.diva-portal.org/smash/get/diva2:1674968/FULLTEXT01.pdf.

Anel, E. (2021). Understanding Social Debt in Software Engineering. [online] Ua.edu. Available at: http://ir.ua.edu/handle/123456789/8278

Ashare, M. (2024). What's the cost of the IT skills gap? IDC says $5.5 trillion by 2026. [online] CIO Dive. Available at: https://www.ciodive.com/news/tech-talent-skills-gaps-cost-trillions-idc/716523/.

Avgeriou, P., Ipek Ozkaya, Chatzigeorgiou, A., Ciolkowski, M., Ernst, N.A., Koontz, R.J., Poort, E. and Shull, F. (2023). Technical Debt Management: The Road Ahead for Successful

Software Delivery. arXiv (Cornell University). doi:https://doi.org/10.1109/icse-fose59343.2023.00007.

Baughan, A., Mercurio, A., Liu, A., Wang, X., Chen, J. and Ma, X. (2023). A Mixed-Methods Approach to Understanding User Trust after Voice Assistant Failures. doi:https://doi.org/10.1145/3544548.3581152.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001). Manifesto for agile software development. [online] Agile Manifesto. Available at: https://agilemanifesto.org/.

Becker, C., Betz, S., Chitchyan, R., Duboc, L., Easterbrook, S.M., Penzenstadler, B., Seyff, N. and Venters, C.C. (2016). Requirements: The Key to Sustainability. IEEE Software, 33(1), pp.56–65. doi:https://doi.org/10.1109/ms.2015.158.

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N. and Venters, C.C. (2015). Sustainability Design and Software: The Karlskrona Manifesto. 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. doi:https://doi.org/10.1109/icse.2015.179.

Berenguer, C., Borges, S., Freire, S., Rios, N., Ramač, R., Nebojša Taušan, Pérez, B., Castellanos, C., Correal, D., Pacheco, A., González López, Manoel Mendonça, Davide Falessi, Seaman, C., Vladimir Mandić, Izurieta, C. and Spínola, R.O. (2023). Investigating the Relationship between Technical Debt Management and Software Development Issues. Journal of Software Engineering Research and Development. doi:https://doi.org/10.5753/jserd.2023.2581.

Besker, T., Ghanbari, H., Martini, A. and Bosch, J. (2020). The influence of Technical Debt on software developer morale. Journal of Systems and Software, [online] 167, p.110586. doi:https://doi.org/10.1016/j.jss.2020.110586.

Besker, T., Martini, A. and Bosch, J. (2018). Managing architectural technical debt: A unified model and systematic literature review. Journal of Systems and Software, 135, pp.1–16. doi:https://doi.org/10.1016/j.jss.2017.09.025.

Boehm, B. and Basili, V.R., 2005. Software defect reduction top 10 list. Foundations of empirical software engineering: the legacy of Victor R. Basili, 426(37), pp.426-4

Borowa, K., Ratkowski, A. and Verdecchia, R. (2025). The technical debt gamble: A case study on technical debt in a large-scale industrial microservice architecture. Journal of Systems and Software, [online] 230, p.112547. doi:https://doi.org/10.1016/j.jss.2025.112547.

Cabllero-Espinosa, E., Carver, J.C. and Stowers, K. (2022). Community Smells -- The Sources of Social Debt: A Systematic Literature Review. [online] arXiv.org. Available at: https://arxiv.org/abs/2209.10671.

Caglayan, D. and Özcan-Top, Ö. (2024). Revisiting Technical Debt Types and Indicators for Software Systems. doi:https://doi.org/10.1145/3605098.3636043.

Chhonker, A. and De Lemos, R. (2019). Microservices Architectures and Technical Debt: A Self-adaptation View. [online] Available at: https://kar.kent.ac.uk/73261/1/msa2019.pdf.

Condori-Fernandez, N., Lago, P., Luaces, M.R. and Places, Á.S. (2020). An Action Research for Improving the Sustainability Assessment Framework Instruments. Sustainability, 12(4), p.1682. doi:https://doi.org/10.3390/su12041682.

Conejero, J.M., Rodríguez-Echeverría, R., Hernández, J., Clemente, P.J., Ortiz-Caraballo, C., Jurado, E. and Sánchez-Figueroa, F. (2018). Early evaluation of technical debt impact on maintainability. Journal of Systems and Software, [online] 142, pp.92–114. doi:https://doi.org/10.1016/j.jss.2018.04.035.

Cunningham, W. (1992). The WyCash portfolio management system. Addendum to the proceedings on Object-oriented programming systems, languages, and applications (Addendum) - OOPSLA ’92. doi:https://doi.org/10.1145/157709.157715.

Dawson, M., Burrell, D.N., Rahim, E. and Brewster, S., 2010. Integrating software assurance into the software development life cycle (SDLC). Journal of Information Systems Technology and Planning, 3(6), pp.49-53.

Dayan-Akman, P., Özcan-Top, Ö. and Temizel, T.T. (2025). People and Management Debt in ML-Integrated Software Projects: Structuring Industry Insights. IEEE Access, 13, pp.137012–137032. doi:https://doi.org/10.1109/access.2025.3595609.

Eskelinen, T., Räsänen, T., Santti, U., Happonen, A., Kajanus, M. (2017). Designing a Business Model for Environmental Monitoring Services Using Fast MCDS Innovation

Support Tools, Technology Innovation Management Review, Vol. 7, Iss. 11, pp. 36-46, doi: 10.22215/timreview/1119

Făgărășan, C., Cristea, C., Cristea, M., Popa, O. and Pîslă, A. (2023). Integrating Sustainability Metrics into Project and Portfolio Performance Assessment in Agile Software Development: A Data-Driven Scoring Model. Sustainability, 15(17), pp.13139–13139. doi:https://doi.org/10.3390/su151713139.

Fairbanks, G. (2020). Ur-Technical Debt. IEEE Software, 37(4), pp.95–98. doi:https://doi.org/10.1109/ms.2020.2986613.

Fatima, I. and Lago, P. (2024). Software Architecture Assessment for Sustainability: A Case Study. Lecture notes in computer science, pp.233–249. doi:https://doi.org/10.1007/978-3-031-70797-1_16.

Flournoy, J.C., Lee, C.S., Wu, M. and Hicks, C.M. (2025). No Silver Bullets: Why Understanding Software Cycle Time is Messy, Not Magic. [online] arXiv.org. Available at: https://arxiv.org/abs/2503.05040.

Fowler, M. (2009). bliki: TechnicalDebtQuadrant. [online] Available at: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

Garousi, G., Garousi, V., Moussavi, M., Ruhe, G. and Smith, B. (2013). Evaluating usage and quality of technical software documentation. Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering - EASE '13. doi:https://doi.org/10.1145/2460999.2461003.

Ghammam, A., Rzig, D.E., Almukhtar, M., Khalsi, R., Hassan, F. and Kessentini, M. (2025). Build Code Needs Maintenance Too: A Study on Refactoring and Technical Debt in Build Systems. 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), [online] pp.616–628. doi:https://doi.org/10.1109/msr66628.2025.00098.

Graham, L., Dzielski, D. and Alshehri, Y. (2025). Architecture Technical Debt Management and Prioritization: A Systematic Mapping Study. Communications in Computer and Information Science, pp.263–281. doi:https://doi.org/10.1007/978-3-031-95127-5_20.

Gruber, M. and Fraser, G. (2022). A Survey on How Test Flakiness Affects Developers and What Support They Need To Address It. [online] arXiv.org. Available at: https://arxiv.org/abs/2203.00483

Guo, Y., Spínola, R.O. and Seaman, C. (2014). Exploring the costs of technical debt management – a case study. Empirical Software Engineering, 21(1), pp.159–182. doi:https://doi.org/10.1007/s10664-014-9351-7.

Happonen, A., Ghoreishi, M. (2022), A mapping study of the current literature on digitalization and industry 4.0 technologies utilization for sustainability and circular economy in textile industries, Lecture Notes in Networks and Systems, Vol. 217, Chapter 63, pp. 697-711, doi: 10.1007/978-981-16-2102-4_63

Heldal, R., Nguyen, N.-T., Moreira, A., Lago, P., Duboc, L., Betz, S., Coroamă, V.C., Penzenstadler, B., Porras, J., Capilla, R., Brooks, I., Oyedeji, S. and Venters, C.C. (2024). Sustainability competencies and skills in software engineering: An industry perspective. Journal of Systems and Software, [online] 211, p.111978. doi:https://doi.org/10.1016/j.jss.2024.111978.

Hou, F. and Jansen, S. (2022). A systematic literature review on trust in the software ecosystem. Empirical Software Engineering, 28(1). doi:https://doi.org/10.1007/s10664-022-10238-y.

Hou, F. and Jansen, S. (2022). A systematic literature review on trust in the software ecosystem. Empirical Software Engineering, 28(1). doi:https://doi.org/10.1007/s10664-022-10238-y.

Huang, C. W., Lin, C., Nguyen, M. K., Hussain, A., Bui, X. T., & Ngo, H. H. (2023). A review of biosensor for environmental monitoring: principle, application, and corresponding achievement of sustainable development goals. Bioengineered, 14(1), 58-80.

IEA (2025). AI is set to drive surging electricity demand from data centres while offering the potential to transform how the energy sector works - News - IEA. [online] International Energy Agency. Available at: https://www.iea.org/news/ai-is-set-to-drive-surging-electricity-demand-from-data-centres-while-offering-the-potential-to-transform-how-the-energy-sector-works.

J. David Morgenthaler, Gridnev, M., Raluca Sauciuc and Bhansali, S. (2012). Searching for build debt: Experiences managing technical debt at Google. CiteSeer X (The Pennsylvania State University). doi:https://doi.org/10.1109/mtd.2012.6225994.

Kaur, H., McDuff, D., Williams, A.C., Teevan, J. and Iqbal, S.T. (2022). 'I Didn't Know I Looked Angry': Characterizing Observed Emotion and Reported Affect at Work. CHI Conference on Human Factors in Computing Systems. doi:https://doi.org/10.1145/3491102.3517453.

Kruglov, A. and Succi, G. (2023). Developing Sustainable and Energy-Efficient Software Systems. SpringerBriefs in computer science. Springer Nature. doi:https://doi.org/10.1007/978-3-031-11658-2.

Leilane Ferreira Ribeiro, de, A., Manoel Mendonça and Rodrigo Oliveira Spínola (2016). Decision Criteria for the Payment of Technical Debt in Software Projects: A Systematic Mapping Study. doi:https://doi.org/10.5220/0005914605720579.

Lenarduzzi, V., Besker, T., Taibi, D., Martini, A. and Arcelli Fontana, F. (2021). A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools. Journal of Systems and Software, 171, p.110827. doi:https://doi.org/10.1016/j.jss.2020.110827.

Levén, W., Broman, H., Besker, T. and Torkar, R. (2024). The broken windows theory applies to technical debt. Empirical software engineering, 29(4). doi:https://doi.org/10.1007/s10664-024-10456-6.

Li, Z., Avgeriou, P. and Liang, P. (2015). A systematic mapping study on technical debt and its management. Journal of Systems and Software, 101, pp.193–220. doi:https://doi.org/10.1016/j.jss.2014.12.027.

Marantos, C., Tsintzira, A.-A., Papadopoulos, L., Ampatzoglou, A., Chatzigeorgiou, A. and Soudris, D. (2020). Technical Debt Management and Energy Consumption Evaluation in Implantable Medical Devices: The SDK4ED Approach. Lecture Notes in Computer Science, pp.348–358. doi:https://doi.org/10.1007/978-3-030-60939-9_25.

Miller, J. (2025). The Hidden Cost of a Messy Dev Environment: Calculating Your Team's 'Configuration Debt'. [online] Medium. Available at:

https://medium.com/@james.miller941/the-hidden-cost-of-a-messy-dev-environment-calculating-your-teams-configuration-debt-b8c91c4dbb41.

Molnar, A.-J. and Motogna, S. (2020). Long-Term Evaluation of Technical Debt in Open-Source Software. arXiv (Cornell University). doi:https://doi.org/10.1145/3382494.3410673.

Moreira, A., Araújo, J., Gralha, C., Goulão, M., Brito, I.S. and Albuquerque, D. (2023). A social and technical sustainability requirements catalogue. Data & Knowledge Engineering, [online] 143, p.102107. doi:https://doi.org/10.1016/j.datak.2022.102107.

Noureddine, A. and Le Goaer, O. (2025). Investigating the Impact of Software Design Patterns on Energy Consumption. 2025 IEEE 22nd International Conference on Software Architecture (ICSA), [online] pp.153–163. doi:https://doi.org/10.1109/icsa65012.2025.00024.

Nugroho, A., Visser, J. and Kuipers, T. (2011). An empirical model of technical debt and interest. Proceeding of the 2nd working on Managing technical debt - MTD ’11. doi:https://doi.org/10.1145/1985362.1985364.

Olsson, J., Risfelt, E., Besker, T., Martini, A. and Torkar, R. (2021). Measuring affective states from technical debt. Empirical Software Engineering, 26(5). doi:https://doi.org/10.1007/s10664-021-09998-w.

Oyedeji, S., Seffah, A. and Penzenstadler, B. (2018). A Catalogue Supporting Software Sustainability Design. Sustainability, 10(7), p.2296. doi:https://doi.org/10.3390/su10072296.

Penzenstadler, B. and Femmer, H. (2013). A generic model for sustainability with process- and product-specific instances. Proceedings of the 2013 workshop on Green in/by software engineering - GIBSE ’13. doi:https://doi.org/10.1145/2451605.2451609.

Pham, Y.D., Abir Bouraffa and Walid Maalej (2020). ShapeRE: Towards a Multi-Dimensional Representation for Requirements of Sustainable Software. pp.358–363. doi:https://doi.org/10.1109/re48521.2020.00048.

Pina, J. (2023). Technical debt prioritization: methods, techniques, and a large exploratory study. Digital Library of Theses and Dissertations (Universidade de São Paulo). doi:https://doi.org/10.11606/t.45.2023.tde-11092023-221224.

Rahman, A., Mahdavi-Hezaveh, R. and Williams, L. (2019). A systematic mapping study of infrastructure as code research. Information and Software Technology, 108, pp.65–77. doi:https://doi.org/10.1016/j.infsof.2018.12.004.

Randell, B. (1979). Software engineering in 1968. pp.1–10. doi:https://doi.org/10.5555/800091.802915.

Restrepo, L., Pardo, C., Aguilar, J., Toro, M. and Suescún, E. (2024). SinSO: An ontology of sustainability in software. Applied Ontology: An Interdisciplinary Journal of Ontological Analysis and Conceptual Modeling, 19(3), pp.231–263. doi:https://doi.org/10.3233/ao-230004.

Rios, N., Mendonça Neto, M.G. de and Spínola, R.O. (2018). A tertiary study on technical debt: Types, management strategies, research trends, and base information for practitioners. Information and Software Technology, 102, pp.117–145. doi:https://doi.org/10.1016/j.infsof.2018.05.010.

Saavedra, N., Mendes, A. and Ferreira, J.F. (2025). Environmental Impact of CI/CD Pipelines. [online] arXiv.org. Available at: https://arxiv.org/abs/2510.26413. Rios, N., Mendes, L., Cerdeiral, C., Magalhães, A.P.F., Perez, B., Correal, D., Astudillo, H., Seaman, C., Izurieta, C., Santos, G. and Oliveira Spínola, R. (2020). Hearing the Voice of Software Practitioners on Causes, Effects, and Practices to Deal with Documentation Debt. Lecture Notes in Computer Science, [online] pp.55–70. doi:https://doi.org/10.1007/978-3-030-44429-7_4.

Sahar Ahmadisakha and Vasilios Andrikopoulos (2024). Architecting for sustainability of and in the cloud: A systematic literature review. Information and software technology, pp.107459–107459. doi:https://doi.org/10.1016/j.infsof.2024.107459.

Sahni, S., Verma, S. and Pratap, R. (2024). Understanding digital transformation challenges for online learning and teaching in higher education institutions: a review and research framework. Benchmarking. doi:https://doi.org/10.1108/bij-04-2022-0245.

Silva, L., Unterkalmsteiner, M. and Wnuk, K. (2023). Towards identifying and minimizing customer-facing documentation debt. arXiv (Cornell University). [online] doi:https://doi.org/10.1109/techdebt59074.2023.00015.

Suvvari, S.K., 2023. Shift Left: Moving the Inclusion of Accessibility Functionalities to the Left in Agile Product Development Life Cycle. Journal of Computational Analysis and Applications, 31(4).

Terrón-Macias, V., Mejia, J. and Terrón-Hernández, M. (2025). A Systematic Literature Review on Technical Debt in Software Development: Types, Tools, and Concerns. International Journal of Combinatorial Optimization Problems and Informatics, 16(4), pp.288–302. doi:https://doi.org/10.61467/2007.1558.2025.v16i4.1150.

Tom, E., Aurum, A. and Vidgen, R. (2013). An exploration of technical debt. Journal of Systems and Software, 86(6), pp.1498–1516. doi:https://doi.org/10.1016/j.jss.2012.12.052.

Trova, G. (2024). Green IT: How green software development can reduce your carbon footprint - SIG. [online] SIG. Available at: https://www.softwareimprovementgroup.com/blog/green-software-development/.

Verdecchia, R. (2021). Architectural Technical Debt: Identification and Management. Vrije Universiteit Amsterdam. [online] doi:https://hdl.handle.net/1871.1/34074ee5-6252-460c-8e78-9f7f93a3b0bb.

Xu, T. and Zhou, Y. (2015). Systems Approaches to Tackling Configuration Errors. ACM Computing Surveys, [online] 47(4), pp.1–41. doi:https://doi.org/10.1145/2791577.

Zazworka, N., Shaw, M., Shull, F. and Seaman, C. (2011). Investigating the impact of design debt on software quality. Proceedings of the 2nd Workshop on Managing Technical Debt. doi:https://doi.org/10.1145/1985362.1985366.

Zhang, W., & Li, G. (2022). Environmental decentralization, environmental protection investment, and green technology innovation. Environmental Science and Pollution Research, 29(9), 12740-12755.

Zhao, Y., De Matteis, T. and Bogner, J. (2025). How Does Microservice Granularity Impact Energy Consumption and Performance? A Controlled Experiment. 2025 IEEE 22nd International Conference on Software Architecture (ICSA), pp.84–95. doi:https://doi.org/10.1109/icsa65012.2025.00018.